

AZ-204

Developing Solutions for Microsoft Azure

Guia de Puntos Clave para el Examen

v3.0 — Preparado por Max Carrión

Dominio	Peso en el examen
Desarrollo de soluciones de proceso de Azure	25-30 %
Desarrollo para Azure Storage	15-20 %
Implementacion de la seguridad de Azure	15-20 %
Supervision, solucion de problemas y optimizacion	5-10 %
Conexion y consumo de servicios de Azure y terceros	20-25 %

Puntuacion minima para aprobar: 700 / 1000

PARTE 1 — Desarrollo de Soluciones de Proceso (25-30%)**1.1 Azure Container Registry (ACR)****Niveles de servicio**

Nivel	Descripción	Uso típico
Basico	Capacidades idénticas a Estandar/Premium pero almacenamiento y throughput limitados	Desarrollo y aprendizaje
Estandar	Mayor almacenamiento y throughput que Basico	Mayoría de escenarios producción
Premium	Mayor throughput + geo-replicación + zone redundancy + Private Link + confianza de contenido	Alta disponibilidad y escenarios distribuidos

⚡ CLAVE

Solo Premium soporta geo-replicación, redundancia de zona y Private Link. Todas las imágenes se cifran en reposo (AES-256). No hay costo adicional por el cifrado.

Admin Account de ACR viene **DESHABILITADO por defecto**. Debes habilitarlo explícitamente si lo necesitas.

Almacenamiento y seguridad ACR

- Cifrado en reposo automático con AES-256 — no requiere configuración adicional
- Geo-replicación (solo Premium): protege ante fallos regionales, mejora latencia de pull/push
- Redundancia de zona (solo Premium): replica en mínimo 3 zonas de disponibilidad
- Almacenamiento regional: datos en la región del registro (excepción: Brasil Sur y Asia SE)
- **Recursos eliminados (repos, imágenes, etiquetas) NO son recuperables**
- Soporta Helm charts e imágenes OCI además de imágenes Docker estándar

ACR Tasks — Automatización de compilaciones

- **Tarea rápida (Quick Task):** compilar e insertar imagen on-demand sin Docker local (equivale a 'docker build + push' en la nube)
- **Tarea por actualización de código fuente:** webhook en GitHub/Azure DevOps activa la compilación
- **Tarea por actualización de imagen base:** recompila automáticamente cuando cambia la imagen padre
- **Tarea programada:** ejecución por temporizador (mantenimiento, pruebas periódicas)
- **Tarea de varios pasos (YAML):** define flujos build → test → push con múltiples contenedores

⚡ CLAVE

Comando clave: `az acr build` ejecuta una Quick Task enviando el contexto a ACR Tasks.

1.2 Azure Container Instances (ACI) y Azure Container Apps (ACA)**Azure Container Instances (ACI)**

- Ejecución de contenedores sin administrar VMs ni orquestadores
- Inicio rápido (segundos) — ideal para tareas cortas, CI, procesamiento por lotes
- **Grupos de contenedores:** múltiples contenedores en el mismo host (comparten red y almacenamiento)
- Montaje de volúmenes: Azure Files, repositorios Git, secretos

- **Políticas de reinicio:** Always / OnFailure / Never
- OS type: Linux (default) o Windows. DNS name label: <label>.<region>.azurecontainer.io
- Se puede usar con Azure Container Apps o AKS para bursting

★ NOTA— Limitaciones importantes de ACI

CRITICO: Los grupos de contenedores SOLO estan disponibles en Linux. **Windows solo permite 1 contenedor por grupo.**

ACI **NO tiene load balancer integrado.** Para balanceo entre contenedores, usar ACA o AKS.

Volumenes en Windows: solo Azure Files mount. Linux soporta todos los tipos (Files, Git, secretos).

Azure Container Apps (ACA)

- Plataforma serverless para microservicios y apps en contenedores — abstrae Kubernetes
- **Escala horizontal basada en KEDA:** HTTP, colas, metricas custom. Puede escalar a CERO instancias
- Ingress integrado (HTTPS), revision management, split traffic (canary/blue-green)
- Entornos: unidad de aislamiento que agrupa multiples Container Apps
- **Dapr:** framework sidecar para comunicacion service-to-service, state, pub/sub

🚫 TRAMPA DEL EXAMEN

Grupos en ACI: Windows = solo 1 contenedor por grupo. Si el examen pide 'multiples contenedores en el mismo host en Windows', ACI no es la respuesta correcta.

restart policy Never: el contenedor NO se reinicia aunque falle. Ideal para jobs de un solo uso, batch processing que no debe repetirse.

1.3 Azure App Service

Planes de App Service

- **Free/Shared (F1, D1):** una sola instancia, no soporta escalado automatico
- **Basic (B1-B3):** escalado MANUAL solamente — **NO soporta escalado automatico**
- **Standard (S1-S3):** escalado automatico basado en reglas. Deployment Slots disponibles
- **Premium V2/V3:** instancias siempre listas, instancias precalentadas, mayor rendimiento

🔑 CLAVE

El escalado automatico requiere plan Standard o superior. F1 y D1 = 1 instancia. B1-B3 = manual. El examen intenta hacer elegir B1 para autoscale.

Escalado automatico — Conceptos clave

- **Scale out/in (horizontal):** agrega/quita instancias — NO cambia CPU/RAM de cada instancia
- Escalar segun metricas: CPU%, memoria%, longitud cola HTTP, longitud cola disco, entrada/salida de datos
- Escalar segun programacion: fija un conteo de instancias en horarios definidos
- **Periodo de recuperacion (cooldown) minimo: 5 minutos** entre eventos de escalado
- **Logica de combinacion de reglas:**
 - **Scale-out:** se dispara si se cumple CUALQUIERA de las reglas (OR)
 - **Scale-in:** se dispara solo si se cumplen TODAS las reglas (AND)

⚡ CLAVE

Duración mínima para análisis: 5 minutos. El intervalo de tiempo BASE es 1 minuto (agregado sobre la duración configurada). Son dos conceptos distintos — el examen los confunde.

Métricas disponibles para escalado automático

Métrica	Que indica
Porcentaje de CPU	Uso de CPU en todas las instancias
Porcentaje de memoria	Uso de memoria — riesgo de fallos si es alto
Longitud de cola de disco	Solicitudes de E/S pendientes — contención de disco
Longitud de cola HTTP	Solicitudes esperando procesamiento — errores HTTP 408 si es alta
Entrada de datos (bytes)	Bytes recibidos en todas las instancias
Salida de datos (bytes)	Bytes enviados desde todas las instancias

Deployment Slots (ranuras de implementación)

- Disponibles en planes Standard y superior
- Cada slot es una app independiente con su propia URL
- Swap: intercambia configuración entre slots (ej: staging → production)
- **Configuraciones NO intercambiables (sticky):** Always On, configuración de escala, Identidades Administradas, restricciones IP, VNET integration, scheduler de WebJobs
- Configuraciones SI intercambiables: Application Settings y Connection Strings (SALVO que estas marcadas como 'deployment slot setting')
- Warm-up automático antes del swap con Always On habilitado

⚡ CLAVE

Durante un swap, las solicitudes al slot de destino se redirigen gradualmente — permite despliegues sin tiempo de inactividad.

Managed Identities NUNCA se intercambian en un swap, ni siquiera si no están marcadas como sticky. Esto es una trampa frecuente.

Auto Swap: NO disponible para Linux Web Apps ni Web App for Containers.

Configuración y settings clave

- **Application Settings:** pares clave-valor que sobrescriben appSettings/connectionStrings del código
- **Connection Strings:** configuradas por tipo (SQLServer, MySQL, PostgreSQL, Custom)
- **Always On:** mantiene la app activa, necesario para WebJobs continuos
- **ARR Affinity:** sesiones de afinidad con instancia específica — **DESACTIVAR si la app es stateless** para mejor distribución de carga
- **HTTPS Only:** fuerza redirección HTTP → HTTPS
- **Minimum TLS Version:** versión mínima TLS para conexiones entrantes

1.4 Azure Functions

Planes de hospedaje

Plan	Escalado	Timeout max	Característica clave
Consumption	Automatico (serverless)	5 min (config hasta 10 min)	Pago por uso — escala a cero. NO soporta VNet
Premium	Automatico con instancias pre-calentadas	Sin limite	Sin cold start, VNet integration, ejecucion larga
Dedicated (App Service)	Manual/automatico segun plan	Sin limite	Reutiliza plan App Service existente
Container Apps	Automatico (KEDA)	Sin limite	Funciones en contenedor con ACA

⚡ CLAVE

Consumption NO soporta VNet integration. Para conectar a recursos en una VNet privada, usar Premium o Dedicated.

Premium: instancias pre-calentadas (pre-warmed) eliminan el cold start. El numero minimo es configurable.

Triggers y bindings

- **Trigger:** define como se invoca la funcion (**solo puede haber 1 por funcion**)
- **Input binding:** datos de entrada adicionales (lectura)
- **Output binding:** datos de salida a otros servicios (escritura)

Triggers comunes:

Trigger	Uso
HttpTrigger	Invocacion via HTTP/REST. Authorization levels: anonymous, function, admin
TimerTrigger	Ejecucion programada (expresion CRON)
BlobTrigger	Se activa al crear/modificar un blob
QueueTrigger	Mensajes en Azure Storage Queue
ServiceBusTrigger	Mensajes en Service Bus Queue o Topic
EventHubTrigger	Eventos en Azure Event Hubs
CosmosDBTrigger	Cambios en el change feed de Cosmos DB

★ NOTA— HttpTrigger — Niveles de autorizacion

anonymous: sin API key — apto para APIs publicas

function: requiere function key — nivel por funcion individual

admin: requiere master key — solo para operaciones administrativas

CRITICO: host.json controla configuracion GLOBAL de todas las funciones. function.json define los BINDINGS de cada funcion individual.

Durable Functions

- Permite escribir flujos de trabajo stateful en Azure Functions
- **Orchestrator function:** coordina la ejecucion (no debe realizar I/O directamente)
- **Activity function:** unidad de trabajo que realiza la tarea
- **Entity function:** estado duradero (contador, acumulador) — actores con estado accedidos via EntityId

★ NOTA— Durable Entities — metodos de acceso desde el Orchestrator

SignalEntityAsync: envia una operacion a la entidad — **fire and forget**. No espera respuesta. Usar cuando no necesitas el resultado

CallEntityAsync: llama a la entidad y **espera la respuesta (await)**. Usar cuando necesitas el valor de retorno

TRAMPA: usar CallEntityAsync cuando no necesitas respuesta bloquea innecesariamente el Orchestrator. El examen pregunta cual usar en cada escenario de codigo

Patrones clave:

- **Function Chaining:** $A \rightarrow B \rightarrow C$ en secuencia
- **Fan-out/Fan-in:** llama multiples actividades en paralelo, espera todas
- **Async HTTP APIs:** cliente inicia tarea larga, consulta estado via polling
- **Monitor:** polling periodico hasta condicion (reemplaza TimerTrigger para esperas variables)
- **Human interaction:** espera evento externo (aprobacion humana)

⚡ CLAVE

Las Orchestrator functions deben ser DETERMINISTAS. No uses `DateTime.Now`, `Guid.NewGuid()` ni llames directamente a APIs externos. Usa `context.CurrentUtcDateTime` en su lugar.

Cada vez que el Orchestrator se reanuda, **replay completo del historial desde el inicio**. Codigo no determinista produce resultados diferentes en cada replay.

🚫 TRAMPA DEL EXAMEN

Output bindings no son transaccionales: si la funcion falla a mitad, algunos outputs pueden haberse ejecutado y otros no.

Consumption timeout: Default = 5 min, maximo configurable en host.json = 10 min. En Premium/Dedicated es ilimitado.

PARTE 2 — Desarrollo para Azure Storage (15-20%)

2.1 Azure Blob Storage

Tipos de recursos

- **Cuenta de almacenamiento:** espacio de nombres unico. URL base: `http://<cuenta>.blob.core.windows.net`
- **Contenedor:** agrupa blobs. Nombres: 3-63 chars, minusculas, numeros y guion, NO puede empezar/terminar con guion, NO guiones consecutivos
- **Blob:** el objeto de datos dentro del contenedor

Tipos de blobs

Tipo	Descripcion	Capacidad / Uso tipico
Block Blob	Datos binarios o texto. Compuesto de bloques gestionables individualmente	Hasta ~190.7 TiB. Archivos generales, imagenes, videos
Append Blob	Optimizado para operaciones de anexion (append)	Logs, auditorias de VMs
Page Blob	Acceso aleatorio a bloques de 512 bytes	Hasta 8 TB. Discos VHD de Azure VMs

Niveles de acceso (Access Tiers)

Nivel	Latencia	Costo almacen.	Costo acceso	Min. dias	Cuando usar
Hot	ms	Alto	Bajo	N/A	Acceso frecuente
Cool	ms	Medio	Medio	30 dias	Acceso ocasional
Cold	ms	Bajo	Alto	90 dias	Acceso raro
Archive	Horas (rehidratacion)	Muy bajo	Muy alto	180 dias	Archivado, backups

⚡ CLAVE

Archive esta OFFLINE — requiere rehidratacion antes de poder acceder. Solo disponible para block blobs individuales, NO a nivel de cuenta.

★ NOTA— Rehidratacion desde Archive — 2 metodos y prioridades

Metodo 1 — Copy Blob: copia el blob archivado a un nuevo blob en nivel Hot/Cool. El blob original QUEDA en Archive.

Metodo 2 — Set Blob Tier: cambia el nivel del mismo blob. Una vez iniciado NO puede cancelarse.

Prioridad Standard: puede tardar hasta 15 horas.

Prioridad High: menos de 1 hora para blobs < 10 GB.

TRAMPA CRITICA: Si usas Set Blob Tier para rehidratar y tienes una lifecycle policy activa, la policy puede **re-archivar el blob automaticamente** porque la 'last modification date' NO cambia durante la rehidratacion.

Lifecycle Management — Politicas

- Definidas en JSON: rules[] → name, enabled, type:Lifecycle, definition → filters + actions
- **Acciones:** tierToCool, tierToCold, tierToArchive, delete, enableAutoTierToHotFromCool
- **Filtros:** blobTypes (requerido), prefixMatch (hasta 10 prefijos), blobIndexMatch (hasta 10 condiciones de tag)
- **Condiciones:** daysAfterModificationGreaterThan, daysAfterCreationGreaterThan, daysAfterLastAccessTimeGreaterThan
- **Si multiples acciones aplican al mismo blob:** lifecycle aplica la accion MAS ECONOMICA (delete < tierToArchive < tierToCool)
- Maximo 100 reglas por politica

⚡ CLAVE

`daysAfterLastTierChangeGreaterThan` solo aplica a tierToArchive y define el minimo de dias en tier caliente antes de volver al archivo.

Seguridad — Cifrado

- Todos los datos cifrados con AES-256 en reposo. Habilitado siempre, no puede desactivarse
- **Claves administradas por Microsoft (default):** Azure gestiona todo
- **Customer-Managed Keys (CMK):** cliente administra claves en Azure Key Vault o Key Vault HSM
- **Customer-Provided Keys (CPK):** cliente incluye la clave en cada request — granular, **solo Blob Storage**. NO aplica a Queues, Tables ni Files
- **Cifrado del lado del cliente (CSE):** cifrado en la app antes de cargar (v2=GCM recomendado, v1=CBC legacy)

★ NOTA— SAS — Shared Access Signatures

Account SAS: delega acceso a uno o mas servicios de Storage (Blob, Queue, Table, File). Firmada con clave de cuenta.

Service SAS: delega acceso a un recurso especifico en UN solo servicio. Soporta Stored Access Policies.

User Delegation SAS: SOLO Blob Storage. Firmada con credenciales de Entra ID — **la mas segura**. No usa la clave de la cuenta.

Stored Access Policy: permite revocar SAS del lado servidor sin cambiar la URL. Solo disponible en Service SAS, NO en Account SAS.

TRAMPA: Para revocar una SAS de Account SAS debes rotar o regenerar la clave de la cuenta. Para Service SAS puedes usar Stored Access Policy.

★ NOTA— SDK .NET — Clases principales de Blob Storage

BlobServiceClient: representa la cuenta de almacenamiento completa (nivel superior)

BlobContainerClient: representa un contenedor especifico

BlobClient: representa un blob especifico

Metadatos de blob: pares clave-valor. Nombres deben ser identificadores C# validos. GetPropertiesAsync() recupera propiedades Y metadatos juntos.

🚫 TRAMPA DEL EXAMEN

Nombres de contenedor: 3-63 chars, solo minusculas, numeros y guion. NO puede empezar ni

terminar con guion. NO guiones consecutivos.

Metadatos con mismo nombre: si envias dos headers de metadatos con el mismo nombre, Blob Storage los concatena con coma y devuelve HTTP 200 (no error).

2.2 Azure Cosmos DB

APIs y modelos de datos

API	Modelo de datos	Cuando usarla
NoSQL (Core SQL)	JSON / documentos	Default — nueva app, sintaxis SQL familiar
MongoDB	BSON / documentos	Migrar apps MongoDB existentes
Cassandra	Tablas / columnar	Migrar apps Cassandra
Gremlin	Grafos	Relaciones complejas entre entidades
Table	Clave-valor (tablas)	Migrar Azure Table Storage

Niveles de consistencia

Nivel (fuerte → debil)	Garantía	Latencia	Disponibilidad
Strong (Fuerte)	Lee siempre la escritura mas reciente	Mayor	Menor
Bounded Staleness	Max K versiones o T tiempo de retraso — SLA de latencia	Alta	Media
Session (DEFAULT)	Consistencia dentro de la sesion del cliente	Baja	Alta
Consistent Prefix	Nunca ve escrituras fuera de orden (pero no garantiza recency)	Baja	Alta
Eventual	Eventual convergencia sin garantias de orden	Menor	Mayor

⚡ CLAVE

Session es el nivel DEFAULT y el mas usado. Strong **NO esta disponible en escrituras multi-region**.

Bounded Staleness: dos parametros — K (numero de versiones) y T (intervalo de tiempo). Es el unico nivel con SLA de latencia garantizado.

Consistent Prefix: NUNCA veras escrituras fuera de orden, pero no garantiza que leas lo mas reciente. El examen confunde esto con Eventual.

★ NOTA— Consistent Prefix vs Eventual — diferencia critica

Consistent Prefix: garantiza **ORDEN** pero NO garantiza recency (puedes leer datos atrasados, pero siempre en orden correcto)

Eventual: NO garantiza **ni orden ni recency**. Puedes ver B antes que A, o no ver ninguno todavia

Regla para el examen: 'nunca fuera de orden' → Consistent Prefix. 'eventual convergencia sin garantias' → Eventual. Ambos tienen alta disponibilidad y baja latencia — esa NO es la diferencia clave

Particionamiento y rendimiento

- **Partition key:** define como se distribuyen los datos. Alta cardinalidad = distribucion uniforme
- **CRITICO:** la partition key es **INMUTABLE** — no puede cambiarse despues de crear el contenedor. Si eliges mal, debes migrar los datos
- **Request Units (RU/s):** moneda de Cosmos DB. Una lectura de 1KB = 1 RU. Una escritura de 1KB ≈ 5 RU
- **Autoscale:** escala entre 10% y 100% del max RU configurado. Si configuras 1000 RU max, el minimo sera 100 RU (no 0)
- Stored procedures, triggers y UDFs: ejecutan en la misma particion logica (ACID dentro de particion)
- **Change Feed:** stream de cambios (inserts y updates) — **NO incluye deletes** (usar soft delete para capturarlos)

★ NOTA— Change Feed Processor — 4 componentes

Contenedor supervisado: donde estan los datos que se monitorean

Contenedor de concesion (Lease): almacena el estado del procesamiento — coordina multiples instancias

Instancia de proceso: el host que ejecuta el procesador (VM, App Service, Azure Function)

Delegado: el codigo que procesa cada lote de cambios

Modelo de datos jerarquico

- Cuenta → Base de datos → Contenedor → Items
- **TTL a nivel contenedor:** define el default de expiracion en segundos
- **TTL a nivel item:** sobreescribe el default del contenedor. **TTL = -1 en item = NUNCA expira**, aunque el contenedor tenga TTL configurado
- Indexing: por defecto todos los campos se indexan automaticamente. Se puede personalizar

⚡ CLAVE

Stored procedures SOLO dentro de una particion logica. No pueden cruzar particiones. Si necesitas transaccion entre 2 partition keys diferentes, no puedes usar stored procedures.

Autoscale ≠ Serverless: Autoscale va de 10% a 100% del max. Serverless paga por consumo exacto. Serverless NO tiene RU garantizados.

PARTE 3 — Implementacion de Seguridad de Azure (15-20%)

3.1 Microsoft Entra ID — Autenticacion y Autorizacion

Flujos de OAuth 2.0 y OpenID Connect

- **Authorization Code Flow:** apps web con servidor backend. Intercambia codigo por tokens
- **PKCE:** extension del auth code flow para SPAs y apps mobiles (sin client secret). NO es un flujo separado
- **Client Credentials Flow:** daemon/servicios sin usuario. Solo tokens de app (no delegados)
- **On-Behalf-Of (OBO):** API intermedia llama a otra API en nombre del usuario original
- **Device Code Flow:** dispositivos sin browser (IoT, CLI, smart TVs)
- **Implicit Flow:** OBSOLETO — no usar en apps nuevas. Reemplazado por Auth Code + PKCE
- **ROPC (Resource Owner Password Credentials):** el mas inseguro. Solo para sistemas legacy. NO recomendado

★ NOTA— Cliente Publico vs Cliente Confidencial

Cliente Publico: SPA, Mobile, Desktop. **NO puede guardar secretos de forma segura.** Usar `PublicClientApplicationBuilder` en MSAL.NET

Cliente Confidencial: Web App, Web API, Daemon. Puede guardar client secret o certificado. Usar `ConfidentialClientApplicationBuilder`

Certificado > Client Secret: certificado es mas seguro que client secret porque no viaja por la red en el intercambio

★ NOTA— Consentimiento incremental y de administrador

Consentimiento dinamico/incremental: puedes solicitar permisos adicionales a medida que el usuario usa mas features, sin pedir todo al inicio

Consentimiento de administrador: permisos de alto privilegio requieren aprobacion de un admin. Si usas consentimiento dinamico con permisos admin, debes registrar TODOS los permisos en el portal de todas formas

Conditional Access puede interrumpir flujos silenciosos (OBO). Tu app debe manejar el 'claims challenge' y redirigir al usuario a autenticacion interactiva

Tokens

- **Access Token:** JWT que autoriza el acceso a recursos. Vida corta (~1 hora)
- **Refresh Token:** permite obtener nuevos access tokens sin login. Vida larga
- **ID Token:** contiene informacion de identidad del usuario (OpenID Connect)
- **Claims:** pares clave-valor dentro del JWT (oid, tid, roles, scp, etc.)
- **Scopes — Delegated:** actuan en nombre del usuario (usuario debe estar logueado)
- **Scopes — Application:** actuan como la app (no hay usuario — daemon/servicio)

Registro de aplicaciones

- **Tenant:** 'common' o 'consumers' para multi-tenant, tenant ID especifico para single-tenant
- **Client ID:** identificador publico de la app (GUID)
- **Client Secret o Certificate:** credencial de la app (certificate = mas seguro)
- **Redirect URI:** URL donde Entra ID envia el codigo de autorizacion
- **API Permissions:** Delegated = en nombre del usuario, Application = solo app

- **Admin consent:** permisos de alto privilegio requieren aprobacion del admin

⚡ CLAVE

MSAL es la libreria recomendada. Maneja cache de tokens, renovacion y los flujos OAuth automaticamente. Azure AD Graph esta DEPRECADO — siempre usar Microsoft Graph.

🚫 TRAMPA DEL EXAMEN

PKCE no es un flujo independiente: es una EXTENSION del Authorization Code flow para clientes publicos que no pueden guardar client secret.

Permisos delegados vs application: Delegated necesita usuario activo. Application permissions son para daemons/servicios. Confundirlos lleva a errores de autorizacion.

3.2 Managed Identities (Identidades Administradas)

Tipos y diferencias

Caracteristica	System-Assigned	User-Assigned
Creacion	Como parte del recurso Azure	Recurso Azure independiente
Ciclo de vida	Se elimina con el recurso	Ciclo de vida independiente
Compartir entre recursos	No — solo 1 recurso	Si — multiples recursos pueden compartirla
Caso de uso	Carga de trabajo en un solo recurso	Multiples recursos con mismos permisos, o permisos preconfigurados

⚡ CLAVE

Usar MI elimina la necesidad de credenciales en el codigo. Se recomienda DefaultAzureCredential que detecta automaticamente el tipo de credencial.

MI es un tipo de Service Principal administrado por Azure. La diferencia clave: Azure gestiona automaticamente las credenciales — no hay secretos ni certificados que rotar manualmente.

Flujo de autenticacion (System-Assigned)

- Azure Resource Manager habilita la identidad → crea Service Principal en Entra ID
- **IMDS endpoint:** <http://169.254.169.254/metadata/identity/oauth2/token> (solo accesible desde dentro del recurso)
- El codigo solicita token al IMDS → Entra ID devuelve JWT
- El codigo usa el JWT para llamar al recurso de Azure destino

DefaultAzureCredential — Orden de autenticacion

1. Variables de entorno (AZURE_CLIENT_ID, AZURE_CLIENT_SECRET, AZURE_TENANT_ID)
2. Identidad administrada (si la app corre en Azure con MI habilitada)
3. Visual Studio (credenciales del IDE en desarrollo)
4. Azure CLI (az login)
5. Azure PowerShell (Connect-AzAccount)
6. Explorador interactivo (deshabilitado por defecto)

Comandos CLI clave

- Crear VM con MI sistema: `az vm create --assign-identity --role contributor`
- Asignar MI sistema a VM existente: `az vm identity assign -g <rg> -n <vm>`
- Crear MI usuario: `az identity create -g <rg> -n <name>`
- **Roles requeridos para MI usuario:** Colaborador de maquina virtual + Operador de identidad administrada

🚫 TRAMPA DEL EXAMEN

Si eliminas un recurso con System-Assigned MI: la MI se elimina automaticamente. PERO si la MI estaba en un Key Vault access policy, el acceso se rompe de inmediato.

MI no soporta todos los servicios: solo servicios que soportan autenticacion de Entra ID. Verificar compatibilidad antes de asumir que funciona.

3.3 Azure Key Vault

Tipos de objetos

- **Secrets:** cadenas de texto plano (connection strings, passwords, API keys). SI exportables
- **Keys:** claves criptograficas (RSA, EC). Operaciones: encrypt, decrypt, sign, verify, wrapKey, unwrapKey. NO exportables en Premium/HSM
- **Certificates:** certificados X.509. Incluye la clave privada asociada. Auto-renovacion configurable

🚫 CLAVE

No guardes claves privadas como Secrets: usa el tipo Keys (con HSM en Premium) para que no sean exportables. Secrets son texto plano exportable.

Niveles de servicio

- **Standard:** secretos, claves software-protected
- **Premium:** agrega soporte para Hardware Security Modules (HSM) — claves no exportables
- **Managed HSM:** HSM dedicado para control completo de claves

Acceso y autenticacion

- Modelo de acceso: plano de gestion (admin del vault) + plano de datos (acceso a secretos/claves/certs)
- **Access Policies (modelo heredado):** permisos a nivel de vault para identidades
- **RBAC (modelo recomendado):** roles estandar de Azure (Key Vault Secrets Officer, Key Vault Reader, etc.)
- **Soft Delete:** retiene objetos eliminados por 7-90 dias (configurable). Habilitado por defecto
- **Purge Protection:** impide eliminar definitivamente objetos durante el periodo de retencion. Requerido para CMK con Cosmos DB/Storage. **UNA VEZ HABILITADA NO PUEDE DESACTIVARSE**

🚫 CLAVE

Para acceder desde codigo: usar DefaultAzureCredential + SecretClient/KeyClient. URI del vault: <https://<nombre>.vault.azure.net/>

Purge Protection es IRREVERSIBLE. Una vez habilitada no puede deshabilitarse. El examen pregunta las implicaciones.

Best Practices

- Una vault por aplicacion por entorno (dev/staging/prod) para aislamiento
- Habilitar diagnostics logging para auditar accesos
- Rotacion de secretos automatica con Event Grid + Azure Functions
- Usar referencias a Key Vault en App Service/Functions para no hardcodear URIs

★ NOTA— Key Vault Firewall y Private Endpoint

Firewall de Key Vault: permite restringir acceso solo a IPs especificas o VNets. Habilitar en la configuracion de Networking del vault

Private Endpoint: asigna una IP privada dentro de tu VNet al vault — el trafico nunca sale a internet publico. Opcion mas segura para entornos corporativos

Cuando usar cada uno: Firewall + Service Endpoint = acceso desde VNet sin IP publica. Private Endpoint = aislamiento completo, trafico 100% privado

TRAMPA: si habilitas el firewall y olvidas agregar la IP de tu App Service o la VNet, la app pierde acceso al vault y falla en runtime — error 403 Forbidden

3.4 Azure App Configuration

Características principales

- Almacen centralizado para configuracion de aplicaciones y feature flags
- Complementa Key Vault: App Config para configuracion no sensible, Key Vault para secretos
- **Etiquetas (labels):** misma clave con diferentes valores por entorno (dev/staging/prod)
- **Snapshot point-in-time:** reproducir configuracion en un momento dado — permite rollback de config
- **Sentinel key pattern:** cambio dinamico de configuracion sin reiniciar la app
- **Feature Management:** activar/desactivar funcionalidades sin redespargar codigo

⚡ CLAVE

Las referencias a Key Vault en App Configuration NO almacenan el secreto — solo almacenan la URI del secreto en Key Vault. La app necesita permisos propios para leer el KV en tiempo de ejecucion.

PARTE 4 — Supervision, Solucion de Problemas y Optimizacion (5-10%)

4.1 Azure Monitor y Application Insights

Componentes de Azure Monitor

- **Metricas:** valores numericos en el tiempo (CPU%, requests/sec, latencia). Retencion: 93 dias
- **Logs (Log Analytics):** datos estructurados/semi-estructurados con Kusto Query Language (KQL)
- **Alertas:** reglas que disparan acciones (email, webhook, runbook) cuando una condicion se cumple
- **Workbooks:** dashboards interactivos que combinan metricas, logs y textos
- **Insights:** experiencias pre-construidas (Application Insights, Container Insights, etc.)

Application Insights

- SDK-based (instrumentacion de codigo) o auto-instrumentacion (sin codigo para App Service, Functions, etc.)
- **Telemetria:** requests, dependencies, exceptions, traces, custom events, custom metrics
- **Correlation:** trace distribuido mediante W3C Trace Context (operationId, parentId)
- Mapa de aplicacion: visualizacion de dependencias entre componentes
- Availability tests: ping tests y multi-step web tests desde ubicaciones globales
- Smart Detection: deteccion automatica de anomalias en rendimiento y fallos
- **Live Metrics Stream:** visualizacion en tiempo real sin latencia de ingestion

★ NOTA— Tipos de Sampling en Application Insights

Adaptive (default en SDK): ajusta automaticamente el porcentaje segun el volumen de telemetria

Fixed rate: tu defines el porcentaje fijo de telemetria a capturar

Ingestion sampling: ocurre en el portal/backend, no en el SDK. No ahorra costos de red

TRAMPA: el sampling puede hacer que NO veas TODOS los errores. Para capturar el 100% de excepciones, configura sampling = 100% o excluye exceptions del sampling

⚡ CLAVE

Connection String vs Instrumentation Key: usar **siempre Connection String**. La Instrumentation Key sola esta en desuso.

Para tracing distribuido entre microservicios: AMBOS servicios deben estar instrumentados y usar W3C Trace Context. Si uno no esta instrumentado, pierdes el correlation completo.

KQL — Consultas clave

- `requests | where resultCode == '500' | summarize count() by bin(timestamp, 1h)`
- `dependencies | where duration > 1000 | order by duration desc`
- `exceptions | summarize count() by type, outerMessage`
- `customEvents | where name == 'Purchase' | extend props = customDimensions | project timestamp, props`

Log Analytics Workspace

- Centraliza datos de Application Insights, Azure Monitor, Security Center, etc.
- **Retencion:** 30 dias default gratuito — configurable hasta 730 dias (pago). Importante para compliance/auditoria

- **Workspace-based AppInsights:** recomendado. Classic AppInsights esta deprecado

PARTE 5 — Conexion y Consumo de Servicios (20-25%)

5.1 Azure API Management (APIM)

Componentes principales

- **API Gateway:** punto de entrada para todas las llamadas. Aplica politicas
- **Developer Portal:** portal auto-generado para documentacion y pruebas de APIs
- **Management Plane:** interfaz de administracion (Azure Portal, ARM, REST API)
- **Backend:** servicio al que APIM proxea las solicitudes
- **Productos:** agrupan una o mas APIs con politicas de acceso y cuotas. Las subscriptions son a PRODUCTOS, no directamente a APIs

Políticas (Policies) — Orden de ejecucion

- **inbound:** autenticacion, rate limiting, transformar request, cache check
- **backend:** modificar URL de backend, set-backend-service
- **outbound:** transformar response, agregar headers, aplicar cache
- **on-error:** manejar errores, devolver respuestas personalizadas

★ NOTA— Elemento `<base />` — CRITICO

Sin `<base />` en la seccion inbound de una politica de operacion: las politicas de nivel API y Global **NO se ejecutan**. Esto puede hacer que `validate-jwt` se salte para esa operacion

Los 4 niveles de scope de politicas: Global (todas las APIs) → Workspace → API → Operacion

cache-lookup debe estar en INBOUND. cache-store debe estar en OUTBOUND. Poner cache-store en inbound no hace nada

Políticas comunes:

- **rate-limit-by-key:** limita llamadas por clave (IP, subscription, etc.)
- **quota-by-key:** cuota total de llamadas en un periodo
- **cache-lookup / cache-store:** cachear respuestas (lookup en inbound, store en outbound)
- **validate-jwt:** validar y verificar tokens JWT (va en INBOUND, antes del backend)
- **set-header:** agregar/modificar headers
- **rewrite-uri:** transformar la URL antes de enviar al backend
- **mock-response:** devuelve respuesta mock sin llamar al backend
- **forward-request:** enviar la solicitud al backend (politica final en backend)

✂ CLAVE

Las politicas se definen en XML y pueden usar expresiones de politica (C# simplificado) encapsuladas en `@(expresion)`.

validate-jwt va en inbound: valida tokens ANTES de que la solicitud llegue al backend.

Niveles de servicio APIM

Tier	Características clave
Consumption	Serverless, pago por llamada, SIN SLA garantizado (0%), limitado en politicas. NO usar en produccion critica

Developer	Solo para desarrollo/pruebas, NO para produccion, tiene portal developer
Basic	Produccion de baja escala, sin VNET
Standard	Produccion, VPN externa
Premium	Multi-region, VNET interna/externa, alta disponibilidad, zona de disponibilidad

★ NOTA— Subscription Keys en APIM

Header: Ocp-Apim-Subscription-Key (o como query param api-key)

Cada suscripción tiene Primary y Secondary key. Para rotación sin downtime: (1) cambiar clientes a Secondary, (2) regenerar Primary, (3) volver clientes a Primary

🚫 TRAMPA DEL EXAMEN

<base /> ausente: si una política de operación no tiene <base /> en inbound, las políticas de nivel API y Global no se ejecutan para esa operación.

Consumption tier: 0% SLA. No soporta VNet integration. No usar en producción para APIs críticas.

5.2 Azure Event Grid

Modelo de eventos

- **Publisher:** quien genera el evento (Azure Storage, IoT Hub, custom source)
- **Topic:** endpoint donde los publishers envían eventos. System topics (recursos Azure), Custom topics, Partner topics (SaaS de terceros: Auth0, Salesforce)
- **Subscription:** configuración que define que eventos van a que handler, con filtros
- **Handler:** quien recibe y procesa el evento (Azure Function, Webhook, Service Bus, Event Hubs, etc.)

🔑 CLAVE

Event Grid sigue el modelo push. Garantiza entrega **al menos una vez** — puede entregar el mismo evento MAS DE UNA VEZ. Tu handler debe ser IDEMPOTENTE.

Orden de eventos NO garantizado. Si necesitas orden, usar Event Hubs (por partición) o Service Bus (con sesiones).

Soporta Event Grid schema Y **CloudEvents 1.0 schema** (estándar CNCF).

Estructura del evento

- **id:** identificador único del evento
- **source:** recurso que generó el evento
- **type:** tipo de evento (ej: Microsoft.Storage.BlobCreated)
- **data:** payload del evento específico del tipo
- **time:** timestamp del evento
- **dataVersion:** versión del esquema del payload

Filtrado de suscripciones

- Por tipo de evento: `includedEventTypes: ['Microsoft.Storage.BlobCreated']`
- Por asunto: `subjectBeginsWith`, `subjectEndsWith`
- Filtrado avanzado: operadores sobre campos del data payload (`NumberGreaterThan`, `StringContains`, etc.)

Patrones de uso y DLQ

- Reaccionar a eventos de Azure (Blob creado, recurso modificado)
- Desacoplar servicios con arquitectura event-driven
- **Dead Letter**: eventos no entregados van a una Storage Account configurada
- **Retry**: politica de reintento exponencial hasta 24 horas

5.3 Azure Event Hubs

Conceptos fundamentales

- Servicio de ingesta de datos masivos (Big Data streaming): millones de eventos/segundo
- Compatible con Apache Kafka (sin cambios de codigo en clientes Kafka)
- Protocolo AMQP 1.0 como protocolo nativo
- Event Hub = registro distribuido de solo anexion, dividido en particiones

Componentes clave

- **Namespace**: contenedor de administracion para uno o mas Event Hubs
- **Event Hub**: canal de transmision (equivalente a un topic de Kafka)
- **Particion**: segmento independiente del log. El numero se define al crear el Hub y **NO puede cambiarse despues**
- **Consumer Group**: vista logica del Event Hub para un conjunto de consumidores. 1 Consumer Group por aplicacion consumidora
- **Offset**: posicion de un consumidor dentro de una particion
- **Checkpoint**: marca el ultimo offset procesado exitosamente

★ NOTA— Retention Period — dato critico

Standard tier: 1-7 dias (default 1 dia).

Premium/Dedicated: hasta 90 dias.

CRITICO: si el consumidor tarda mas que el retention period, **los eventos se pierden permanentemente**.

Unidades de rendimiento

- **Throughput Unit (TU) — Plan Estandar**: 1MB/seg o 1000 eventos/seg entrada, 2x de salida
- **Processing Unit (PU) — Plan Premium**: mayor rendimiento
- **Event Hubs Capture**: captura automatica a Blob Storage o Data Lake en formato Apache Avro

Procesamiento con EventProcessorClient

- Recomendado para produccion (vs EventHubConsumerClient para escenarios simples)
- Distribucion automatica de particiones entre instancias del procesador
- **Checkpoint Store**: necesario para balanceo y recuperacion. Tipicamente usa Azure Blob Storage
- Patron consumidor particionado: cada particion procesada por un solo consumidor en un momento dado

⚡ CLAVE

El numero de particiones define el maximo de consumidores paralelos por consumer group.

No puedes tener mas consumidores activos que particiones.

Orden garantizado SOLO dentro de una particion. Entre particiones, no hay garantia de orden.

TRAMPA DEL EXAMEN

Particiones inmutables: el numero de particiones se define al crear el Event Hub y NO puede cambiarse. Elegir bien desde el inicio.

Endpoint Kafka vs AMQP: la compatibilidad con Kafka usa un endpoint diferente al AMQP. No son intercambiables en la URL de conexion.

5.4 Azure Service Bus

Tiers de Service Bus

Tier	Capacidades
Basic	SOLO Queues. NO tiene Topics ni Subscriptions
Standard	Queue + Topics + Subscriptions. Duplicate detection. Shared capacity
Premium	Todo de Standard + mayor throughput, VNet integration, geo-disaster recovery, zona de disponibilidad. Capacidad dedicada

CLAVE

Topics NO disponibles en tier BASIC. Si el examen pregunta que tier usar para Topics/Subscriptions → Standard o Premium.

Entidades de mensajería

- **Queue:** mensajería FIFO punto a punto. Un solo consumidor por mensaje
- **Topic + Subscriptions:** mensajería pub/sub. Múltiples suscriptores reciben el mismo mensaje
- **Reglas de suscripción:** filtros SQL o correlación para seleccionar que mensajes recibe cada suscripción

Características clave

- **At-least-once delivery (PeekLock):** mensaje se bloquea al leer, se confirma (Complete) o abandona (Abandon)
- **Sesiones:** entrega ordenada de mensajes relacionados por session ID
- **Dead-Letter Queue (DLQ):** mensajes que no pueden procesarse. Max delivery count default = 10 intentos
- **Deferred messages:** procesamiento postergado con SequenceNumber
- **Scheduled messages:** entrega en fecha/hora futura
- **Duplicate detection:** detecta y descarta mensajes duplicados por MessageId. Ventana default = 10 min. Solo Standard y Premium
- **TTL:** expiración de mensajes si no son procesados

Proceso de 2 pasos (PeekLock)

- 1. ReceiveMessages: obtiene el mensaje y lo hace invisible para otros consumidores (lock)
- 2a. Complete/DeleteMessage: confirma el procesamiento exitoso, elimina el mensaje
- 2b. Abandon: devuelve el mensaje a la cola para ser reintentado

⚡ CLAVE

Si el código falla entre paso 1 y 2, el lock expira y otro consumidor puede procesar el mensaje. Garantiza que ningún mensaje se pierda.

Message lock renewal: si el procesamiento tarda más que el lock timeout, debes renovar el lock con `RenewMessageLockAsync()` para evitar que otro consumidor tome el mensaje.

🚫 TRAMPA DEL EXAMEN

ReceiveAndDelete vs PeekLock: `ReceiveAndDelete` elimina el mensaje en el mismo paso de lectura. Si el código falla, el mensaje SE PIERDE. `PeekLock` es más seguro.

Topics en Basic tier: no disponibles. Solo Queues.

Comparacion Service Bus vs Event Grid vs Event Hubs

Servicio	Patron	Uso típico	Orden
Service Bus	Mensajería empresarial (Queue/Topic)	Workflows, transacciones, desacople con garantías	FIFO con sesiones
Event Grid	Eventos reactivos (push)	Reaccionar a cambios de estado en Azure o custom	No garantizado
Event Hubs	Streaming de Big Data	Telemetría, logs, analytics en tiempo real	Por partición

5.5 Azure Storage Queues

Características y limitaciones

- Parte de Azure Storage Account — no requiere servicio separado
- **Tamaño máximo de mensaje: 64 KB**
- Número de mensajes: ilimitado (hasta el límite de la cuenta de almacenamiento)
- **Visibilidad default al leer: 30 segundos** — configurable
- Proceso de 2 pasos: `ReceiveMessages` (invisible) → `DeleteMessage` (eliminar)
- `UpdateMessage`: actualiza contenido + extiende visibilidad (útil para checkpointing)
- `PeekMessages`: lee sin hacer invisible (no consume el mensaje)
- **`GetProperties` devuelve `ApproximateMessagesCount`** — puede ser MAYOR al número real. No usar para lógica crítica

⚡ CLAVE

Storage Queue vs Service Bus: Storage Queue es más simple y económica (mensajes < 64KB, sin orden garantizado, sin DLQ nativa). Service Bus: mensajes más grandes, DLQ, sesiones, duplicate detection, scheduled messages, transacciones.

5.6 Microsoft Graph API

Conceptos básicos

- **URL base:** `https://graph.microsoft.com/{version}/{recurso}`
- **Versiones:** `v1.0` (producción) y `beta` (preview — NO usar en producción)
- Autenticación: OAuth 2.0 con tokens de Entra ID (MSAL recomendado)
- **Permisos Delegated:** en nombre del usuario. Application: solo app, sin usuario

- **Azure AD Graph esta DEPRECADO:** siempre usar Microsoft Graph

Operaciones CRUD

Metodo HTTP	Operacion
GET	Leer datos de un recurso — sin body
POST	Crear recurso o ejecutar accion — requiere body JSON
PATCH	Actualizar recurso con nuevos valores (parcial) — requiere body JSON
PUT	Reemplazar recurso completo — requiere body JSON
DELETE	Eliminar recurso — sin body

- **OData query params:** `$filter`, `$orderBy`, `$select`, `$top`, `$skip`, `$expand`, `$search`

SDK de Microsoft Graph (.NET)

- **Microsoft.Graph:** modelos y builders para v1.0
- **Microsoft.Graph.Beta:** para el endpoint beta
- **GraphServiceClient:** instancia unica para toda la app, acepta `IAuthenticationProvider`
- **Paginacion:** usar `Pageliterator` para iterar todas las paginas
- **Batch requests:** hasta 20 requests individuales en una sola llamada HTTP

⚡ CLAVE

Throttling en Graph: puede devolver HTTP 429 (Too Many Requests). Implementar retry con exponential backoff usando el header `Retry-After`.

REFERENCIA RAPIDA — Datos Numericos Clave

Los numeros exactos suelen aparecer en preguntas del examen:

Servicio / Concepto	Dato / Limite
Blob Storage — Block Blob max size	~190.7 TiB
Blob Storage — Page Blob max size	8 TB
Blob Storage — Cool tier min retention	30 dias
Blob Storage — Cold tier min retention	90 dias
Blob Storage — Archive tier min retention	180 dias
Archive — rehidratacion Standard	hasta 15 horas
Archive — rehidratacion High priority	menos de 1 hora (blobs < 10 GB)
Lifecycle policy — max reglas por politica	100
Container name — longitud	3-63 caracteres
Event Hubs — TU estandar entrada	1 MB/seg o 1000 eventos/seg
Event Hubs — TU estandar salida	2 MB/seg o 2000 eventos/seg
Event Hubs — retention Standard	1-7 dias (default 1 dia)
Event Grid — reintento max tiempo	24 horas
Service Bus — DLQ max delivery count default	10 intentos
Service Bus — duplicate detection window default	10 minutos
Storage Queue — tamano max mensaje	64 KB
Storage Queue — visibilidad default al leer	30 segundos
App Service Autoscale — periodo recuperacion min	5 minutos
App Service Autoscale — duracion analisis min	5 minutos
Functions Consumption — timeout default	5 minutos (max configurable 10 min)
Key Vault Soft Delete — retencion min	7 dias
Key Vault Soft Delete — retencion max	90 dias
Azure AD Access Token — vida tipica	~1 hora
Microsoft Graph — max requests en batch	20 requests
Microsoft Graph — version produccion	v1.0
APIM — politicas orden ejecucion	inbound → backend → outbound → on-error
ACR — niveles de servicio	Basic, Standard, Premium
Cosmos DB — consistencia default	Session
Cosmos DB — lectura 1KB en RU	1 RU
Cosmos DB — escritura 1KB en RU (aprox)	~5 RU
Cosmos DB — autoscale minimo RU	10% del max configurado

TIPS FINALES PARA EL EXAMEN

Patrones que el examen frecuentemente evalua

- Elegir el nivel de ACR correcto (geo-replicacion → Premium, produccion normal → Standard)
- Elegir el tier de Blob Storage (frecuente → Hot, 30d → Cool, 90d → Cold, 180d → Archive)
- **Archive requiere rehidratacion (horas)** — Standard hasta 15h, High priority menos de 1h para <10GB
- **Consistencia Session es el DEFAULT de Cosmos DB**
- Distinguir Event Grid (reactivo/push) vs Event Hubs (streaming/Big Data) vs Service Bus (mensajería empresarial)
- **DefaultAzureCredential:** orden de autenticacion y que funciona tanto en desarrollo como en produccion
- Managed Identity: System-Assigned = unico por recurso y se elimina con el. User-Assigned = reutilizable
- App Service escalado: scale-out usa OR para reglas de escalar, AND para reglas de reducir
- Lifecycle policy: si multiples acciones aplican al mismo blob, se aplica la mas economica
- **Key Vault: Purge Protection es IRREVERSIBLE.** CMK requiere Purge Protection
- **APIM: <base /> es CRITICO** — sin el, politicas de niveles superiores no se ejecutan
- Durable Functions Orchestrator: determinista — no usar DateTime.Now, Guid.NewGuid() ni I/O directo
- Event Hubs: numero de particiones = maximo de consumidores paralelos por consumer group
- **PeekLock en Service Bus / 2-step delete en Storage Queue:** garantiza que mensajes no se pierdan ante fallos
- **Service Bus Basic tier: SOLO Queues.** Para Topics/Subscriptions → Standard o Premium
- SAS: User Delegation SAS es la mas segura (usa Entra ID, no la clave de la cuenta)

Errores comunes a evitar

- **NO confundir Event Grid (eventos, reactivo) con Event Hubs (streaming, volumen masivo)**
- **NO usar Implicit flow** de OAuth en apps nuevas (obsoleto — usar Auth Code + PKCE)
- **NO hardcodear Connection Strings** — usar App Configuration + referencias a Key Vault
- **NO asumir que App Service Basic soporta escalado automatico** — requiere Standard o superior
- **NO confundir Managed Identity con Service Principal** — MI es un tipo de SP administrado por Azure
- **NO usar ApproximateMessagesCount de Storage Queue para logica critica** — puede ser mayor al real
- **NO usar Microsoft Graph API endpoint beta en produccion**
- **NO olvidar que la partition key de Cosmos DB es IMMUTABLE** despues de crear el contenedor
- **NO asumir que Consumption plan de Functions soporta VNet** — requiere Premium o Dedicated
- **NO usar Azure AD Graph** — esta DEPRECADO. Usar Microsoft Graph
- **NO olvidar que el numero de particiones de Event Hubs es IMMUTABLE** despues de crear el hub

!Buena suerte en el examen AZ-204!

Puntuacion minima para aprobar: 700 / 1000